Pattern Recognition 180 (2026) 113900

Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/pr

Stratify: Rethinking federated learning for non-IID data through balanced sampling

Hui Yeok Wong, Chee Kau Lim *, Chee Seng Chan

Faculty of Computer Science and Information Technology, Universiti Malaya, Kuala Lumpur, 50603, Malaysia

ARTICLE INFO

Dataset link: https://github.com/HuiYeok1107/Stratify

Keywords:
Federated learning
Non-IID data
Label skew
Feature skew

ABSTRACT

Federated Learning (FL) on non-independently and identically distributed (non-IID) data remains a critical challenge, as existing approaches struggle with severe data heterogeneity. Current methods primarily address the effects of non-IID data by applying incremental adjustments to Federated Averaging (FedAvg), rather than directly resolving its inherent design limitations. Consequently, performance significantly deteriorates under highly heterogeneous conditions, as the fundamental issue of imbalanced exposure to diverse class and feature distributions remains unresolved. This paper introduces *Stratify*, a novel FL framework designed to systematically manage class and feature distributions throughout training, effectively tackling the root cause of non-IID challenge. Inspired by classical stratified sampling, our approach employs a Stratified Label Schedule (SLS) to ensure balanced exposure across labels, significantly reducing bias and variance in aggregated gradients. Complementing SLS, we propose a label-aware client selection strategy, restricting participation exclusively to clients possessing data relevant to scheduled labels. Additionally, *Stratify* incorporates a fine-grained, high-frequency update scheme, accelerating convergence and further mitigating data heterogeneity. To uphold privacy, we implement a secure client selection protocol leveraging homomorphic encryption, enabling precise global label statistics without disclosing sensitive client information. Extensive evaluations on MNIST, CIFAR-10, CIFAR-100, Tiny-ImageNet, COVTYPE, PACS, and Digits-DG demonstrate that *Stratify* attains performance comparable to IID baselines, accelerates convergence, and reduces client-side computation compared to state-of-the-art methods, underscoring its practical effectiveness in realistic federated learning scenarios. Code is available at https://github.com/HuiYeok1107/Stratify.

1. Introduction

Federated learning (FL) has emerged as a promising paradigm for distributed machine learning that preserves data privacy by enabling clients to collaboratively train a global model without sharing their raw data [1]. However, a persistent challenge in FL is that client data are often non-independently and identically distributed (non-IID). In practical settings, clients typically possess heterogeneous datasets with imbalanced class distributions or varying feature characteristics. Such data heterogeneity leads to imbalanced local updates, which, when aggregated, bias the global model and slow down convergence [2].

Existing research has developed various algorithms to address non-IID challenge. The common approaches used include: (i) cluster similar clients for collaboration [3–6], (ii) weighted local model parameters aggregation [7–9], (iii) data augmentation [10–12], (iv) apply correction term to restrict local deviation from global model [13–15], and (v) ensemble-based approach [16,17]. These approaches primarily focus on avoiding collaboration among non-IID clients to align with IID assumption [18] or applying patches on FedAvg to deal with diverged local updates. However, such methods address only the non-IID data effects rather than directly targeting the root cause: the uneven exposure of the global model to diverse label and feature distributions.

In this work, we introduce *Stratify*, a novel framework for federated learning that addresses the non-IID challenge by rethinking the training process. Our method is built on the concept of a Stratified Label Schedule (SLS), which draws inspiration from classical stratified sampling [19]. By constructing a balanced and randomized schedule of labels based on controlled repetition frequencies, *Stratify* ensures that the global model is systematically exposed to all classes, thereby mitigating bias and reducing the variance of the aggregated gradients. Our theoretical analysis, grounded in the principles of stratified sampling [19] and variance decomposition [20], demonstrates that this balanced update scheme yields an unbiased gradient estimator with improved stability.

* Corresponding author.
E-mail addresses: s2124360@siswa.um.edu.my (H.Y. Wong), limck@um.edu.my (C.K. Lim), cs.chan@um.edu.my (C.S. Chan).

https://doi.org/10.1016/j.patcog.2026.113900
Received 3 May 2025; Received in revised form 2 January 2026; Accepted 28 April 2026
Available online 11 May 2026

Complementing the SLS, our framework employs a label-aware client selection mechanism that restricts participation to those clients possessing the relevant data for the designated label. Additionally, we propose a novel learning strategy, in which the global model is sequentially updated using individual samples or small batch samples (see Section 3.3). This fine-grained update process accelerates convergence and further alleviates the adverse effects of data heterogeneity. To ensure that these guided updates are implemented in a privacy-preserving manner, we develop a secure client selection protocol that leverages a masked label representation based on the Cheon-Kim-Kim-Song (CKKS) [21] homomorphic encryption (HE) scheme. This approach enables accurate global label statistics to be computed without exposing clients' sensitive information.

Our contributions are fourfold.

- *Stratify* introduces SLS and a corresponding label-aware client selection to achieve balanced and robust model updates under non-IID conditions.
- Our novel learning strategy provides high-frequency, fine-grained updates that lead to faster convergence.
- Our secure implementation, which utilizes masked label representation with CKKS, ensures that these benefits are realized without compromising client privacy.
- Extensive experiments on benchmark datasets (i.e., MNIST [22], CIFAR-10 [23], CIFAR-100 [23], Tiny-ImageNet [24,25], COVTYPE [26], PACS [27], and Digits-DG [28]) demonstrate that *Stratify* achieves performance close to the IID baseline, converges in fewer rounds, and achieves lower overall local computation workload per client, compared to state-of-the-art methods.

2. Related work

2.1. Client clustering

Various algorithms have been developed to address non-IID data challenge in FL. The clustering approach groups clients with similar data using similarity measure to enable collaboration among homogeneous clients. Briggs et al. [3] and CFMTL [6] use the updated local model parameters with a distance metric to find similar clients. FEDCOLLAB [29] compares client data distributions by training a global discriminator where a balanced accuracy of around 50% indicates similar client data. FedDrift [4] measures the maximum loss degradation when applying local models to each other's data. Clients with loss values below a threshold are merged. PACFL [5] uses truncated SVD to extract key features of each client's data, and clusters clients with similar data based on principal angles spanned by these vectors. However, clustering approach is ineffective for small participating clients and each holds diverse data. The likelihood of finding clients with similar data can be slim as the similarity measures used in these research identify homogeneous clients based on their overall datasets [8]. Furthermore, these methods have to maintain multiple global models which increases complexity and limits generalization across clients.

2.2. Weighted aggregation

Some works tackled non-IID issue by minimizing the variance of client updates through weighted contributions. FedAMP [7] and APPLE [8] maintain a personalized model for each client on the server, and update each model through weighted combination of model parameters from other clients. FedAMP measures each pairwise client parameters using Cosine similarity and assigns higher weight to models that are similar during aggregation. APPLE uses the directed relationship (DR) matrix to quantify the influence of other client models on a client model. Each client DR matrix is adaptively updated during training, with weights adjusted based on the gradients of the loss function. If another client's model negatively impacts the client, the corresponding weight in the DR matrix moves closer to zero or becomes negative to minimize the harmful influence. However, this approach faces the same limitations as clustering approach as it relies on similarity metric to guide collaboration. In highly diverse client datasets, this approach will struggle to assign meaningful weights.

2.3. Local model regularization

FedProx [13], SCAFFOLD [15], and DOMO [14] modify the aggregation process to stabilize learning on non-IID data. FedProx adds a proximal regularization on the local model against the global model to keep the updated parameter close to the global model. SCAFFOLD estimates the difference in update directions for each client and the global model. This difference captures the drift of the client parameters from the global model, which is then used to correct the local update. DOMO adjusts the local model momentum of the client to the direction of global momentum. However, in cases of extreme data heterogeneity, the differences between local and global model can be too large, making regularization alone insufficient to bridge the divergence effectively.

2.4. Data augmentation

Data augmentation methods in FL aim to mitigate non-IID effects by enhancing local data diversity. FRAug [10] tackles feature skew by training a shared generator to produce client-agnostic embeddings. These embeddings are transformed by a local Representation Transformation Network (RTNet) into client-specific residuals, which are then used to augment real feature representations to enhance model robustness against feature distribution shifts. Li et al. [11] generates synthetic data by extracting essential class-relevant features using class activation map. Before data generation, real features are adjusted along the hard transformation direction. This ensures that the synthetic data contains less detailed information and becomes less similar to the real data. The synthetic data is then collected by server and shared with clients to combine with local real samples. However, while data augmentation enhances data diversity, it may not fully capture the complexities and nuances of real data [30], especially when synthetic data must avoid closely mimicking real samples in order to protect sensitive patterns. Therefore, a method that can directly utilize the diverse real data of clients for training is more practical.

2.5. Local model ensemble

The Ensemble approach combines the predictions from multiple specialized models to mitigate non-IID and enhance generalization ability across diverse client distributions. Zhao et al. [16] groups clients with similar initial gradients into clusters, each collaboratively train a global model. During inference, the predictions from these individual models are combined using weighted coefficients to generate the final prediction. Fed-ensemble [17] trains an ensemble of K models. For each round, each client receives one of the K models for local training and sends the updated model to server for aggregation. After training, all K models are sent to clients and prediction on new data is obtained by averaging the prediction output of all models. However, this approach requires client to hold multiple models locally, which is not storage efficient for resource-constrained devices.

2.6. Client selection

Client selection methods often use clustering to group similar clients, and instead of collaborating within cluster, they sample representative clients from different clusters to enhance model generalization. Fraboni et al. [31], cluster similar clients using their sample size or representative gradients which indicate the difference between their local updates and the global model. Clients are sampled from these

different clusters for collaboration, ensuring diverse representation during aggregation. FedSTS [32] uses compressed gradient to filter out redundant information in the raw gradients for better grouping clients into strata. During client selection, clients within each stratum with the higher gradient norms are given higher probabilities of being selected, ensuring that the most significant updates are included. However, in cases of extreme data diversity among clients, these methods will fail as a client does not clearly belong to a single cluster due to diverse data characteristics. This results in poorly defined groups that lead to a suboptimal client selection and ultimately degrading the model performance.

Despite extensive research on improving FL for non-IID data, existing approaches still face limitations including reliance on potential noisy similarity metrics for clustering, and the increased complexity and storage inefficiency from maintaining multiple models. In contrast, our proposed solution effectively learn on non-IID client data without being restricted by these limitations. We present *Stratify* in the next section.

3. *Stratify* Framework

Federated learning on non-IID data often leads to imbalanced local updates and biased global models when using conventional aggregation techniques (e.g., FedAvg). To address these challenges, our proposed *Stratify* framework introduces a guided training mechanism that ensures balanced exposure to all class labels during global model updates. Central to our approach is the Stratified Label Schedule (SLS), a mechanism inspired by classical stratified sampling that produces unbiased gradient estimates with reduced variance. The following subsections detail the construction of the SLS, its theoretical foundation, and how it informs both client selection and model training.

3.1. Stratified label schedule

To mitigate the adverse effects of data heterogeneity, the server aggregates the unique labels from all participating clients and constructs a balanced schedule by repeating each label l a controlled number of times according to f_l. We define the Stratified Label Schedule (SLS) as

$$\text{SLS} = \text{Shuffle}\left(\bigcup_{l\in\mathcal{L}} f_l\{l\}\right),$$

where $\mathcal{L}$ is the set of all unique labels, $f_l\{l\}$ denotes that label l is repeated f_l times, and f_l can be set uniformly or proportionally to a capped global count (see Section 4.1) for each label l to prevent overrepresentation.

Shuffling randomizes the order so that each training cycle exposes the global model to a diverse and balanced mix of classes. In each training iteration, the server selects the next label l from the SLS and assigns training tasks exclusively to those clients that possess samples with label l.

3.1.1. Theoretical analysis: Stratified sampling and variance reduction

Inspired by classical stratified sampling where a population is divided into homogeneous subgroups (strata) and samples are drawn from each to reduce estimator variance. We view the SLS as a mechanism for enforcing balanced label representation. In our context, each label l forms a stratum. Let g_l denote the gradient computed from local sample with label l. The aggregated gradient update is then computed as

$$\hat{g} = \frac{1}{|\text{SLS}|}\sum_{l\in\text{SLS}} g_l,$$

By the linearity of expectation, we have

$$\mathbb{E}[\hat{g}] = \sum_{l\in\mathcal{L}} P(l)\,\mathbb{E}[g_l],$$

where the probability-proportional-to-size for label l is defined as:

$$P(l) = \frac{f_l}{\sum_{l\in\mathcal{L}} f_l},$$

and with f_l denoting the frequency of label l in SLS. By enforcing a uniform frequency across all labels $l \in \mathcal{L}$, $P(l)$ becomes uniform. As a result, each label contributes equally to the aggregated gradient. This confirms that SLS yields an unbiased and balanced gradient estimator by constructing an equal representation across all labels.

Furthermore, by the law of total variance:

$$\text{Var}(\hat{g}) = \mathbb{E}\left[\text{Var}(\hat{g} \mid l)\right] + \text{Var}\left(\mathbb{E}[\hat{g} \mid l]\right).$$

The term $\text{Var}\left(\mathbb{E}[\hat{g} \mid l]\right)$ reflects the variance due to imbalanced contributions across labels. Enforcing balanced sampling via the SLS minimizes this term relative to naive sampling, leading to reduced overall variance and more stable convergence in non-IID settings.

3.2. Client selection based on the SLS

Building on the SLS mechanism, the server employs a label-aware client selection strategy. For each training iteration, after extracting the next label l from the SLS, the server selects only those clients whose local datasets contain samples with label l. This targeted selection ensures that the gradient update for that iteration is computed solely from data corresponding to l, thereby preserving the balanced representation and variance reduction benefits established in Section 3.1. Algorithm 1 and 2 detail the client selection process. *Stratify* supports two client selection strategies:

(i) **Uniform Client Selection:** An equal probability of client selection that maximizes label privacy. This strategy relies only on masked label availability, ensuring strong label privacy without requiring the server to access any label-size-related metadata. Uniform selection works well in most non-IID scenarios, particularly the label skew setting and mild feature skew cases. The actual label identities of clients are masked using abstract placeholders, ensuring that the true semantic label values remain hidden from server. This allows server to perform label-aware selection without compromising label privacy (see Section 4.1).

(ii) **Weighted Client Selection:** An adjusted selection probabilities based on the proportion of each client's label count relative to the global label count, computed using HE. This strategy mitigates the risk of overfitting to the clients feature domains with disproportionately large data during the later training iterations, by enforcing a consistent exposure to diverse client domains. While it introduces a slight relaxation in label privacy, the obfuscated label identifiers ensure that the label proportions are untraceable to the actual categories they represent. This trade-off allows for better handling of severe feature heterogeneity, while maintaining adequate label privacy safeguard.

In Algorithm 1, the $chunkSize$ parameter is introduced to process labels from SLS in chunks, allowing the server to group multiple labels and assign them consecutively to the same client if available. This parameter also effectively controls the model's exposure frequency to diverse client data between chunks, which is especially useful for feature-skew scenarios. Whereas the $batchSize$ parameter in Algorithm 2, controls the number of labels used in each global batch update.

3.3. Model training procedure

Conventional methods (e.g., FedAvg) rely on aggregating coarse-grained updates after clients train on their entire local datasets for multiple local epochs. Our proposed learning approach presents a novel contribution to the federated learning paradigm by facilitating frequent, fine-grained updates from clients to the global model. This enhances the global model's ability to adapt quickly and converge more effectively, particularly in heterogeneous settings.

Algorithm 1 Single-Sample Learning Client Selection.

Input: $Placeholder_{unique}$, P_{chunk} (see Algo. 3, line 6), $clientPool$ (see Section 4.1)
Output: C_{select}

$C_{select} \leftarrow [\,]$
for $p \in Placeholder_{unique}$ **do**
  $availClients_p \leftarrow$ **Select** clients from $clientPool_p$ using uniform or weighted client selection
end for
for $(i,\ p)$ in enum(P_{chunk}) **do**
  for $c_j \in availClients_p$ **do**
    $next\_i \leftarrow i$
    while True **do**
      $next\_i \leftarrow \min(next\_i + 1,\ chunkSize - 1)$
      if $c_j \in availClients_p$ **of** $P_{chunk}[next\_i]$ **then**
        $conseqLen_{c_j} \leftarrow conseqLen_{c_j} + 1$
        if $next\_i = chunkSize - 1$ **then**
          break out from while loop
        end if
      else
        break out from while loop
      end if
    end while
  end for
  $C_{select}[i : i + conseqLen_{c_j} + 1] \leftarrow$ **Select** any one c_j with $conseqLen_{c_j} = \max(conseqLen)$
  continue outer for loop at $i \leftarrow i + conseqLen_{c_j} + 1$
end for

Algorithm 2 Batch-Data Learning Client Selection.

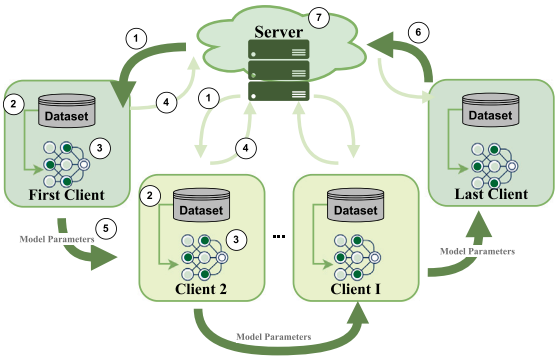
Input: $Placeholder_{unique}$, P_{batch} (see Algo. 4, line 4), $clientPool$ (see Section 4.1)
Output: C_{select}

$C_{select} \leftarrow [\,]$
for $p \in Placeholder_{unique}$ **do**
  $availClients_p \leftarrow clientPool_p$
end for
for $p \in$ set(P_{batch}) **do**
  $N_p \leftarrow$ Freq(p, P_{batch})
  $C^p_{elem} \leftarrow$ **Select** N_p number of clients from $availClients_p$ using uniform or weighted clients selection
end for
for $(i,\ p)$ in enum(P_{batch}) **do**
  $C_{select}[i] \leftarrow C^p_{elem}[0],\ C^p_{elem} \leftarrow C^p_{elem}[1:]$
end for

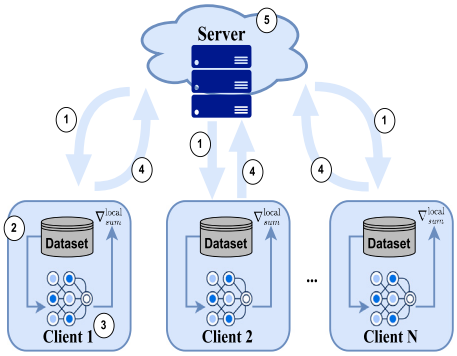
Following client selection based on the SLS, *Stratify* utilizes two complementary training strategies to update the global model:

(i) **Single-Sample Learning:** In this learning setting (see Fig. 1(a) and Algorithm 3), the global model is passed sequentially from one selected client to the next. At each step, a client performs an update using a single sample corresponding to the label (or consecutive labels) indicated by the SLS. While our approach follows a sequential model-passing structure similar to Sequential Federated Learning (SFL) [33], our method introduces two novel aspects: (1) model updates are routed through a client selection mechanism rather than randomly, and (2) clients perform multiple lightweight updates using small samples each time, rather than full local data training at once. These key aspects significantly enhance the global model's adaptability and performance in non-IID settings.

(ii) **Batch-Data Learning:** In the batch-data learning setting (see Fig. 1(b) and Algorithm 4), a mini-batch of samples is aggregated from the selected clients based on the SLS. Each client computes gradients using summed losses over sampled data corresponding to the assigned labels, and these gradients are then aggregated at the server — by summing and normalizing with the batch size — to update the global model. This gradient treatment, in conjunction with our guided training and label-aware client selection, ensures that the aggregated gradients remain balanced, unbiased, and exhibit lower variance. Security measures including homomorphic encryption, secure multi-party computation, and differential privacy, can be applied for secure aggregation. We leave the exploration of security aspects as an important direction for future work, since our research primarily focuses on introducing novel guided training approach to address non-IID data.



(a) Single-sample learning



(b) Batch-data learning

Fig. 1. Overview of *Stratify* Training Process. (a) Single-sample learning: (a1) Send initialized global model parameter (only first client), masked labels to train, and next client address to current client, (a2) Convert masked label to real label and sample the required data, (a3) Update model sequentially on the required label data, (a4) Signal server to send next masked labels to train to next client, (a5) Send updated model to next client, Repeat step a1 to a5 until all masked labels to train are pulled from SLS, (a6) Send the final updated model to server once the last client completes its training and (a7) Broadcast the received global model to all clients. (b) Batch-data learning: (b1) Send masked labels to train to all selected clients for the current batch, (b2) Convert masked label to real label and sample the required data, (b3) Compute summed gradient, (b4) Return summed gradient, and (b5) Compute a unified gradient to update global model by summing up the clients' summed gradients and then dividing it with the batch size, and send the updated model to all selected clients in next batch. Repeat step b1 to b5 until all masked labels to train are pulled from SLS.

Our *Stratify* learning approach offers three key benefits:

- Fine-Grained, Immediate Updates: The model receives frequent, incremental updates that immediately incorporate each sample's contribution, ensuring that no single client's skewed data disproportionately influences the training.
- Mitigation of Non-IID Effects: By updating the model on a per-sample, label-specific basis, the sequential update mechanism directly counters biases from heterogeneous data distributions. As

demonstrated in Section 3.1.1, these updates yield an unbiased gradient estimator with reduced variance, leading to improved stability and faster convergence. Our learning approach is particularly advantageous in scenarios with extreme non-IID data, clearly distinguishing *Stratify* from traditional FL protocols.

- Effective participation of clients with limited data: Our learning approach ensures that even a small contribution of clients is meaningfully integrated by accumulating its summed gradients as part of the contribution for a batch or directly incorporating its small update into the single-sample learning model. This addresses a key limitation of conventional FL, which requires clients to possess a sufficient amount of data for effective local training [2].

Together, these two training procedures provide flexibility. Single-sample learning is well-suited in online or streaming scenarios where data arrives sequentially and must be processed within certain time period due to storage constraint. Conversely, batch-data learning is effective when clients have a large and static local data. They can also be used in hybrid, where the global model is first pretrained on large local datasets using batch updates. Once deployed, the model can continuously learn from the new data using single-sample updates, allowing it to adapt over time in dynamic environments.

Algorithm 3 Single-Sample Per Iteration Model Training.

Server Input: Clients' addresses, $chunkSize$, initial θ_{global}, $clientPool$
Server Output: Final trained θ_{global}
Client Input: θ_{global} from server or previous client
Client Output: Updated θ_{global}, $P_{unavail}$

Server-side
$SLS \leftarrow$ **Generate** Stratified Label Schedule
$queue \leftarrow [\,]$
while $|SLS|$ **or** $|queue| \neq 0$ **do**
  if $|queue| < 2$ **and** $|SLS| \neq 0$ **then**
    $P_{chunk} \leftarrow SLS[0:chunkSize]$, $SLS \leftarrow [chunkSize:]$
    $C_{select} \leftarrow$ **Run** Algo. 1 on P_{chunk}
    Group P_{chunk} placeholders by consecutive identical client ID length in C_{select}
    $queue \leftarrow$ **Append** each group as training task containing selected client and placeholders to train details
  end if
  $task \leftarrow$ **Pop** first training task in $queue$
  $task \leftarrow$ **Add** next client address from $queue[0]$ client detail **if** $|queue| \neq 0$ **else** server address
  $P_{unavail} \leftarrow$ **Send** $task$ to client for training
  Add each $p \in P_{unavail}$ into SLS at random position
  Remove client from $clientPool_p$ **for** $p \in$ set$(P_{unavail})$
end while
Broadcast final trained θ_{global} received from the last client to all clients
Client-side
$\theta_{local} \leftarrow \theta_{global}$
for $p \in placeholdersToTrain$ **require in** $task$ **do**
  $x, y \leftarrow$ **Extract** one $(x, y) \in D_{local}$ where $y = y_{real}$ with $p \mapsto y_{real}$ **if** available, **else add** p to $P_{unavail}$
  $\theta_{global} \leftarrow \theta_{local} - \eta \cdot \nabla_{\theta_{local}} \mathcal{J}(f_{\theta_{local}}(x), y)$
end for
Signal server to send training task to next client in queue
Send updated θ_{global} to next destination address
Return $P_{unavail}$ back to server ($P_{unavail} = []$ if no unavailable placeholder)

3.4. Custom batch normalization layer

In non-IID FL settings, Batch Normalization (BN) [34] suffers from mismatched batch statistics across clients. This discrepancy can lead to unstable updates and degraded performance [35]. To cater for the use of BN layer in *Stratify*, our BN solution is inspired by the solution (FedTAN) proposed in [35] for FedAvg algorithm. FedTAN modifies the forward and backward propagation of BN layer to compute the global batch statistics and gradients across clients. During the forward pass, the local batch mean and variance statistics are sent to the server, which averages them to obtain the global batch statistics and send back to clients to proceed with the subsequent computations. Similarly, during the backward pass, the average gradients of these statistics are computed by the server and returned back to clients. However, since the forward-pass μ mean and σ^2 variance statistics:

$$\mu = \frac{1}{m}\sum_{i=1}^{m} x_i, \quad \sigma^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu)^2,$$

and the backward-pass terms in the derivative of the loss $\mathcal{J}$:

$$G = \sum_{i=1}^{m} \frac{\partial \mathcal{J}}{\partial \hat{x}_j}, \quad G_x = \sum_{i=1}^{m} \hat{x}_j \frac{\partial \mathcal{J}}{\partial \hat{x}_j},$$

which require batch information involve only sum-based computation. The sums over smaller subsets across clients can be accumulated first, then only divide by the scaling factor. This can better resolve the mismatch in *Stratify*, instead of averaging the local batch statistics and gradients across clients to get the global statistics, as the assigned placeholder subsets across clients represent the full batch when combined. Privacy measures such as homomorphic encryption can be applied to shield the batch information from server.

Algorithm 4 Batch-Data Per Iteration Model Training.

Server Input: Clients' addresses, $batchSize$, initial θ_{global}, $clientPool$
Server Output: Final trained θ_{global}
Client Input: θ_{global} from server
Client Output: $grad^{local}_{sum}$, $P_{unavail}$

Server-side
$SLS \leftarrow$ **Generate** Stratified Label Schedule
while $|SLS| \neq 0$ **do**
  $P_{batch} \leftarrow SLS[0:batchSize]$, $SLS \leftarrow SLS[batchSize:]$
  $C_{select} \leftarrow$ **Run** Algo. 2 on P_{batch}
  for all $c_i \in$ set(C_{select}), **concurrently do**
    $P^{c_i}_{subset} \leftarrow [P_{batch}[j] \mid C_{select}[j] = c_i]$
    $P_{unavail} \leftarrow$ **Send** Training Request containing θ_{global}, $P^{c_i}_{subset}$ to c_i
    Remove client from $clientPool_p$ **for** $p \in$ set$(P_{unavail})$
    Rerun Algo. 2 on $P_{unavail}$ and **Repeat** line 6-9 until available clients are found for all $p \in P_{batch}$
  end for
  $grad_{sums} \leftarrow$ **Signal** awaiting clients to start Local Training
  $grad_{avg} \leftarrow \frac{1}{|P_{batch}|}\sum_{i=0}^{|grad_{sums}|} grad_{sums}[i]$
  $\theta_{global} \leftarrow \theta_{global} - \eta \cdot grad_{avg}$
end while
Broadcast final trained θ_{global} to all clients
Client-side
Training Request:
$\theta_{local} \leftarrow \theta_{global}$
for $p_i \in P^{c_i}_{subset}$ **do**
  $X_i, Y_i \leftarrow$ **Extract** one $(x, y) \in D_{local}$ where $y = y_{real}$ with $p_i \mapsto y_{real}$ **if** available, **else add** p_i to $P_{unavail}$
end for
Return $P_{unavail}$ to server ($P_{unavail} = []$ if no unavailable placeholder)
Local Training:
$\mathcal{J}_{sum} \leftarrow \sum_{i=0}^{|Y|} \ell(f_{\theta_{local}}(X_i), Y_i)$
$grad^{local}_{sum} \leftarrow \nabla_{\theta_{local}} \mathcal{J}_{sum}$
Return $grad^{local}_{sum}$ to server

4. Implementation details and communication efficiency

In practical federated learning systems, securing client participation and managing communication overhead are critical to ensuring system effectiveness. In this section, we describe our privacy-preserving client selection protocol and analyze communication costs, providing real-world implementation details with mathematical rigor.

4.1. Secure client selection with masked label representation

To ensure that only appropriate clients participate in each training iteration while preserving the privacy of their local label distributions. We propose a masked label representation approach that leverages homomorphic encryption (specifically, the CKKS scheme). In this method, each client i computes its local label counts $n_i(l)$ for each label l and then masks these labels and counts by encrypting them as $[E(l_i), E(n_i(l))]$ pairs, using the CKKS encryption scheme.

To securely identify all unique labels without exposing the actual label value, the server initializes a list of unique labels $\mathcal{L}_{unique}$ by adding all the encrypted labels from first clients. For each subsequent client i, the server compares its encrypted label $E(l_i)$ with the labels already in $\mathcal{L}_{unique}$ using encrypted subtraction operation:

$$E(\text{Diff}_{ij}) = E(l_i) - E(l_j),\ \forall E(l_j) \in \mathcal{L}_{\text{unique}}.$$

The result of this operation against all unique labels is decrypted by client i. If they are all not 0, $E(l_i)$ is considered unique and added to $\mathcal{L}_{unique}$. Once all unique labels are identified, each unique $E(l)$ is mapped to a randomly generated placeholder p (e.g., ASCII codes) to facilitate the subsequent clients' label-to-placeholder mapping. The mapping process also uses the encrypted subtraction operation by comparing each client's encrypted label $E(l_i)$ with the unique labels in $\mathcal{L}_{unique}$. If a match is found, the corresponding placeholder p is assigned to that client's label.

With all labels mapped to placeholders, the server proceeds to aggregate the encrypted counts $E(n_i(l))$ from all clients to obtain the global count for each placeholder:

$$E(N(p)) = \sum_{i \in C_p} E(n_i(l)),\ \ E(l_i) \mapsto p,$$

where C_p denotes the set of clients with $E(l_i)$ mapped to p. Thanks to the additive homomorphic properties of CKKS, the server can compute $E(N(p))$ without decrypting individual values. Once aggregation is complete, the server, in coordination with the clients (who hold the decryption keys), securely decrypts $E(N(p))$ to reveal the global counts $N(p)$ in a privacy-preserving manner. These counts then inform the frequency f_l used in constructing the SLS (see Section 3.1).

This unified protocol combining masked label representation with homomorphic encryption ensures accurate global label statistics while protecting client privacy. Our use of HE for secure client selection aligns with prior work in [36], demonstrating that carefully designed multiparty HE schemes can provide privacy guarantees and practical performance in federated learning without compromising model utility.

4.2. Communication overhead analysis

Efficient communication is essential for practical federated learning. Let T denote the total communication time per client, R denotes the number of training rounds, C_{update} represents the cost (in seconds or data volume) per model update (weights or gradients) transfer, and f_{freq} represents the frequency of model update transfers per round:

$$T = R \times C_{\text{update}} \times f_{\text{freq}}. \tag{1}$$

Since *Stratify* employs frequent, fine-grained global model updates, f_{freq} is higher in our learning approach compared to conventional methods. Specifically, in our batch-learning approach, the transfer frequency is once per batch iteration, assuming that the client participates in every batch iteration. Whereas in our single-sample approach, the transfer frequency per client is equivalent to the expected amount of local labels contributing to the SLS, when the $chunkSize$ is set to 1. As the $chunkSize$ increases, f_{freq} reduces due to consecutive placeholders being assigned to the same client per training task (see ablation study in Section 5.7).

Although f_{freq} is higher in our learning approach, increasing the number of clients effectively spreads out f_{freq} to other available clients, leading to lower f_{freq} per client. Empirical findings further indicate that our learning approach requires significantly fewer communication rounds (R) to reach convergence compared to all baselines. In the federated learning scenarios where high-bandwidth connections are available, the additional communication overhead is often negligible.

4.3. Integration with stratify training procedures

The practical implementation details described above ensure that the theoretical benefits of our guided training mechanism are preserved during real-world operation. Specifically, the secure client selection protocol with its masked label representation enabled by CKKS homomorphic encryption guarantees that the SLS is constructed accurately without compromising client privacy. The communication overhead analysis demonstrates that, despite the higher update frequency inherent in *Stratify* learning strategy, the overall communication cost is kept manageable due to the reduced number of training rounds.

These implementation aspects bridge the gap between our theoretical framework (Sections 3.1–3.1.1) and practical deployment, confirming that *Stratify* is both effective in theory and viable in real-world federated learning scenarios.

5. Experimental results and discussion

In this section, we present extensive experimental evaluations of our proposed *Stratify* framework on multiple benchmark datasets. We compare *Stratify* with several state-of-the-art methods under diverse non-IID conditions and analyze key aspects such as convergence behavior, accuracy, and communication overhead.

5.1. Experimental setup

Baselines, Datasets, and Evaluation Metric. Baseline FL algorithms, including FedAvg, FedProx, SCAFFOLD, vanilla SFL, and FedTAN are implemented for comparisons. The global model performance is evaluated using the average Top-1 test accuracy across all clients. The algorithms are trained for the maximum epoch used in their respective IID cases, and the epoch that obtains the highest accuracy is recorded. The algorithms are evaluated on datasets including MNIST, CIFAR-10, CIFAR-100, Tiny-ImageNet, and COVTYPE (for label skew) as well as PACS and Digits-DG (for feature skew).

Non-IID Data Partitions. This study follows the non-IID data partitioning methods proposed in [37], where datasets are partitioned based on the number of classes (#C) or domains (#D) assigned to each client. The severity of non-IID increases as the number of assigned classes or domains decreases. The samples of each label are randomly and equally distributed to the clients that hold the label. To simulate imbalanced data distributions, a common challenge in federated learning, the datasets are partitioned using both label-skew schemes and a Dirichlet distribution (with a concentration parameter of 0.5). The Dirichlet distribution is widely used to control the degree of heterogeneity among clients; a lower concentration parameter (0.5) results in highly skewed, non-IID data. This isolates the impact of client heterogeneity without confounding effects from global class imbalance. *Stratify* employs uniform client selection across all non-IID scenarios for the label skew setting. In the feature skew setting, both uniform and weighted client selection strategies are explored, with the better-performing one reported.

Model Architecture, Hardware and Software Environment. For single-sample and batch learning, LeNet is used for MNIST and DigitDG. PACS employs a simple CNN with three convolution-pooling blocks and a fully connected classifier. COVTYPE uses a four-layer MLP with ReLU and dropout, followed by a binary output layer. For batch

Table 1
Averaged Top-1 test accuracy of Federated Learning Algorithms using single-sample per iteration learning (Batch Size 1) with 10 clients. E# denotes the epoch at which the algorithm achieves its highest accuracy. U and W denote the accuracy is achieved using uniform or weighted client selection.

Non-IID	Dataset	Partitioning	FedAvg	FedProx	SCAFFOLD	SFL	*Stratify* (Ours)
Label Skew	MNIST	#C=1	18.71%, E10	12.37%, E14	17.68%, E30	15.19%, E29	**98.80%, E4**
		#C=3	91.67%, E30	85.27%, E29	91.11%, E18	72.52%, E30	**99.06%, E5**
		#C=5	95.42%, E30	96.51%, E30	92.60%, E24	89.30%, E28	**98.95%, E5**
		Dirichlet(0.5)	97.82%, E30	97.19%, E30	98.75%, E27	97.76%, E28	**98.94%, E5**
		IID	98.32%, E30	98.34%, E30	98.71%, E19	98.79%, E5	98.82%, E5
	CIFAR-10	#C=1	11.56%, E7	11.43%, E38	10.46%, E2	10.00%, E1	**81.18%, E29**
		#C=3	42.61%, E50	50.44%, E49	59.12%, E50	30.10%, E42	**80.15%, E22**
		#C=5	66.36%, E50	65.94%, E50	73.83%, E50	60.10%, E46	**80.71%, E30**
		Dirichlet(0.5)	72.37%, E50	66.75%, E50	75.13%, E50	76.38%, E24	**81.06%, E23**
		IID	73.76%, E50	74.28%, E50	74.48%, E50	82.60%, E50	80.15%, E30
	CIFAR-100	#C=10	32.55%, E50	18.77%, E49	27.74%, E50	12.53%, E46	**55.00%, E43**
		#C=30	35.33%, E50	34.92%, E50	32.52%, E49	31.22%, E48	**54.32%, E41**
		#C=50	37.12%, E49	36.57%, E50	33.13%, E50	44.88%, E48	**54.69%, E48**
		Dirichlet(0.5)	38.56%, E50	38.50%, E50	33.76%, E50	51.80%, E45	**54.62%, E45**
		IID	39.6%, E50	39.8%, E50	35.71%, E50	63.22%, E50	54.26%, E50
	COVTYPE	#C=1	58.61%, E22	54.34%, E30	57.07%, E1	57.03%, E1	**87.07%, E30**
		Dirichlet(0.5)	75.07%, E27	66.24%, E28	71.60%, E24	44.10%, E12	**86.19%, E28**
		IID	77.38%, E30	70.64%, E30	60.00%, E30	86.65%, E30	86.94%, E30
Feature Skew	PACS	#D=1	87.13%, E29	92.00%, E30	88.60%, E30	94.93%, E30	**95.24%, E15(U)**
		#D=2	94.65%, E28	95.28%, E30	89.31%, E30	**95.61%, E29**	95.35%, E13(U)
		Dirichlet(0.5)	84.4%, E30	90.74%, E29	85.68%, E30	89.57%, E29	**95.84%, E14(U)**
		IID	95.72%, E30	95.68%, E30	92.57%, E30	95.30%, E30	95.12%, E15
	Digits-DG	#D=1	82.69%, E15	82.56%, E15	**83.04%, E13**	78.60%, E13	82.48%, E13(U)
		#D=2	83.09%, E13	**83.95%, E15**	82.83%, E14	82.14%, E14	82.33%, E14(U)
		Dirichlet(0.5)	82.31%, E15	**83.56%, E15**	82.83%, E15	78.00%, E7	82.6%, E15(U)
		IID	84.44%, E15	83.79%, E15	83.56%, E15	81.79%, E15	82.20%, E15

learning, CIFAR-10 and CIFAR-100 use ResNet-9 with BN, and Tiny-ImageNet uses ResNet-18 with BN. In single-sample learning, CIFAR-10 and CIFAR-100 employ four convolutional blocks, each containing two 2D convolutional layers (CIFAR-100 with LayerNorm), followed by max pooling and dropout layers, and ending with a fully connected classifier. All experiments were conducted on an NVIDIA A100 GPU with 80 GB memory, using Python 3.10 and PyTorch 2.6.0. Each client in the federated learning setup was simulated as a separate process, with communication between clients and the server performed via REST API.

5.2. Impact of non-IID severity on performance

As shown in Tables 1 and 2, as the number of labels (#C) and domains (#D) each client holds decreases, *Stratify* is the only algorithm that consistently achieves accuracy close to IID case in both single-sample and batch-data learning settings, while the accuracy of all baselines declines in most cases. In Dirichlet-based data partition scenarios, *Stratify* consistently outperforms all baselines, particularly in batch-data learning. Its performance remains stable even in the presence of label imbalance. *Stratify*'s resiliency towards varying non-IID severity is attributed to the use of SLS that restricts the global model to be updated following the label sequence defined. It iteratively takes a fine-grained and immediate global update strategy using SLS as its guide, which enables the global model to adapt to the diverse data in a gradual and unbiased manner.

On the contrary, all baselines require clients to train on most or all of their available data at once before sending the update to server. This results in diverged local updates in non-IID settings as each client's update is constructed based on a skewed data distribution. While FedProx and SCAFFOLD introduce mechanisms to mitigate local drift, their correction effectiveness is limited in highly non-IID settings and is inconsistent across the two learning settings and non-IID types. SCAFFOLD, while more effective than FedProx on label-skew data with batch learning setting, struggles in single-sample learning and performs poorly with feature-skew data. This is because SCAFFOLD's control variates can only adjust the gradient drift in clients' updates to ensure update consistency, which does not explicitly align drifted feature representations.

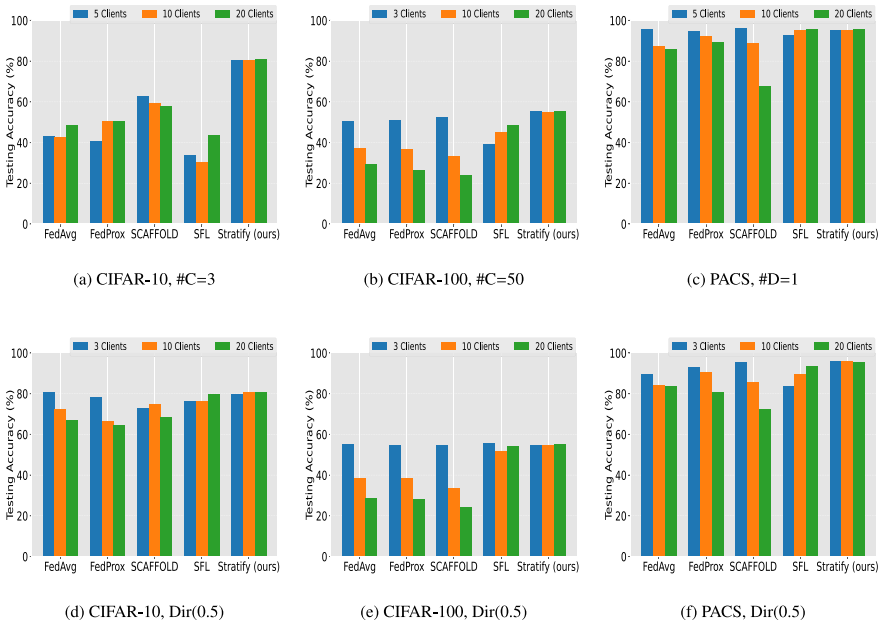
Stratify effectively addresses feature skew by consistently introducing the global model to label data from diverse clients using the client selection mechanisms that designed to ensure an unbiased feature representations learning. FedProx, on the other hand, requires a proper tuning of proximal coefficient based on the degree of data heterogeneity, which is often unknown in practice. As a result, finding a suitable coefficient requires multiple rounds of FL, making the optimization process less efficient with unguaranteed performance improvement on severe data heterogeneity scenarios. These show that correcting local drift is challenging, as its effectiveness depends on the degree of data heterogeneity and a proper parameter tuning. Hence, without addressing the fundamental design limitations of FedAvg in handling non-IID data, these baselines struggle to consistently improve performance across different non-IID scenarios.

5.3. Performance with increasing dataset classes

Among all the evaluated algorithms, *Stratify* is the only algorithm that consistently achieves accuracy similar to its IID case across all datasets with varying class sizes. This demonstrates *Stratify*'s robustness in learning from a broader range of classes by incorporating all the diverse labels in SLS. In contrast, the accuracy gap between FedAvg, FedProx, and SCAFFOLD compared to *Stratify* increases in both single-sample and batch-data label-skew settings as the number of dataset classes increases. This trend can be clearly observed by analyzing the accuracies across datasets partitioned using Dirichlet(0.5) and the least severe #C, both of which exhibit less extreme label-skew to better isolate the effect of increasing dataset classes. SFL, on the other hand, manages to maintain a small accuracy gap in the Dirichlet(0.5) partitioned due to its sequential learning nature, which allows it to progressively learn across variable range of labels. However, this would

Table 2
Averaged Top-1 Test Accuracy of Federated Learning Algorithms using Batch-Data Per Iteration Learning with 10 Clients. E# denotes the epoch at which the algorithm achieves its highest accuracy. U and W denote the accuracy is achieved using uniform or weighted client selection.

Non-IID	Dataset	Partitioning	FedAvg	FedProx	SCAFFOLD	*Stratify* (Ours)
Label Skew	CIFAR-10 (with BN)	#C=1	23.45%, E44	29.38%, E49	18.27%, E36	**90.60%, E8**
		#C=3	62.15%, E39	52.75%, E50	66.72%, E50	**90.45%, E8**
		#C=5	64.03%, E45	71.40%, E50	84.39%, E47	**90.54%, E8**
		Dirichlet(0.5)	81.29%, E42	80.11%, E47	84.54%, E48	**90.27%, E8**
		IID	88.84%, E50	88.71%, E50	87.48%, E50	90.39%, E8
	CIFAR-100 (with BN)	#C=10	36.40%, E50	36.73%, E48	39.10%, E42	**74.45%, E30**
		#C=30	42.39%, E50	42.18%, E50	51.76%, E48	**74.80%, E29**
		#C=50	46.77%, E50	46.80%, E50	61.39%, E49	**74.65%, E30**
		Dirichlet(0.5)	47.57%, E50	46.62%, E50	64.74%, E49	**74.72%, E30**
		IID	48.28%, E50	48.64%, E50	66.19%, E50	74.41%, E30
	Tiny-ImageNet (with BN)	#C=20	14.93%, E70	12.65%, E70	25.88%, E70	**51.09%, E50**
		#C=60	17.86%, E70	17.85%, E70	35.10%, E70	**51.60%, E49**
		#C=100	22.14%, E70	21.36%, E70	41.28%, E70	**50.72%, E49**
		Dirichlet(0.5)	23.10%, E70	23.21%, E70	42.40%, E68	**50.62%, E46**
		IID	35.69%, E70	35.59%, E70	43.77%, E70	51.15%, E50
	COVTYPE	#C=1	57.03%, E1	57.01%, E1	57.31%, E25	**90.19%, E29**
		Dirichlet(0.5)	77.26%, E39	80.23%, E49	78.14%, E46	**89.95%, E27**
		IID	85.60%, E50	85.75%, E50	84.17%, E50	89.48%, E30
Feature Skew	PACS	#D=1	78.69%, E30	85.85%, E29	42.67%, E13	**94.74%, E7(W)**
		#D=2	93.26%, E30	82.57%, E30	92.32%, E30	**94.24%, E7(W)**
		Dirichlet(0.5)	76.04%, E30	74.59%, E30	77.59%, E30	**95.71%, E7(W)**
		IID	93.80%, E30	93.92%, E30	90.88%, E30	94.15%, E7
	Digits-DG	#D=1	78.31%, E15	80.48%, E15	68.94%, E15	**84.75%, E13(U)**
		#D=2	80.92%, E15	79.77%, E15	75.39%, E15	**85.03%, E11(U)**
		Dirichlet(0.5)	77.96%, E14	78.46%, E15	56.02%, E15	**84.73%, E13(U)**
		IID	80.21%, E15	81.60%, E15	74.73%, E15	85.21%, E13



(a) CIFAR-10, #C=3 (b) CIFAR-100, #C=50 (c) PACS, #D=1

(d) CIFAR-10, Dir(0.5) (e) CIFAR-100, Dir(0.5) (f) PACS, Dir(0.5)

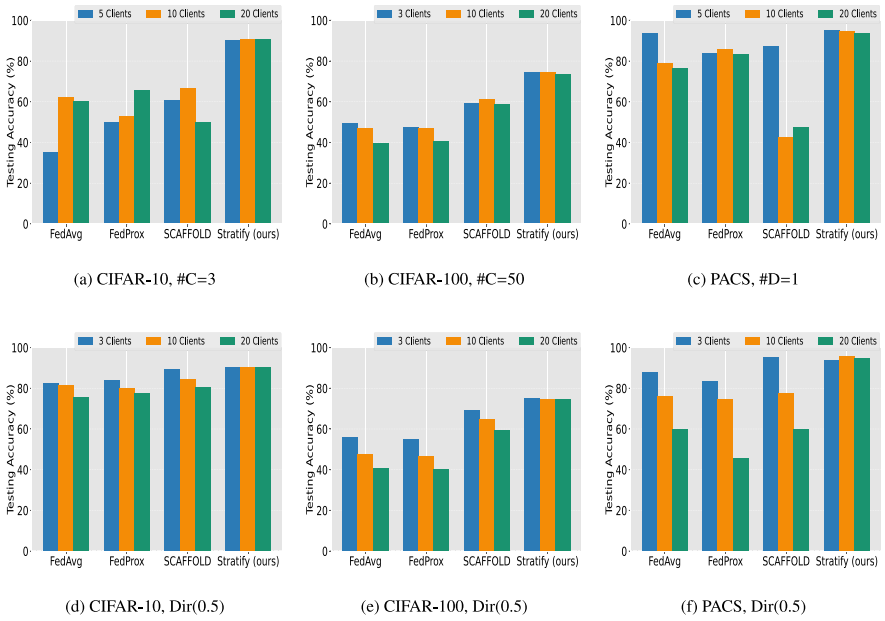
Fig. 2. Comparison of algorithms' performance with increasing client numbers on different datasets and data partitions in single-sample per iteration learning.

require SFL clients to have minimal non-overlapping labels to minimize catastrophic forgetting. In the #C partition type, where clients have larger disjoint label sets, the accuracy gap between SFL and *Stratify* widens as the learned representations from previous clients are progressively overwritten by recent clients with disjoint labels.

Hence, *Stratify* proves to be a more effective approach for scenarios with increasing class diversity, as it consistently maintains high accuracy where the baselines struggle.

5.4. Performance with increasing client size

Figs. 2 and 3 illustrate the effect of increasing client number on algorithms' performance across different datasets and data partitions. In both single-sample and batch-data learning settings, FedAvg, FedProx, and SCAFFOLD generally exhibit a decline in accuracy as more clients participate, with some fluctuations in certain cases. The accuracy decline is due to the reduced local dataset size in the class and domain



(a) CIFAR-10, #C=3 (b) CIFAR-100, #C=50 (c) PACS, #D=1

(d) CIFAR-10, Dir(0.5) (e) CIFAR-100, Dir(0.5) (f) PACS, Dir(0.5)

Fig. 3. Comparison of algorithms' performance with increasing client numbers on different datasets and data partitions in batch-data per iteration learning.

based partitions which constrain per-client learning, and the growing data imbalance in Dirichlet partition which further exacerbating the disparity between local updates. An opposite trend is observed for SFL. Notably, in Dirichlet partition, SFL achieves accuracy comparable to *Stratify* as more clients participate. This is because the data distribution becomes more spread out across clients as the client number increases, and reduces the risk of SFL overfitting due to prolonged training on a single client large data. However, SFL accuracies in the class and domain based partitions are still lower than *Stratify* due to catastrophic forgetting caused by disjoint client labels.

All in all, *Stratify* maintains stable performance regardless of increasing client numbers because its global model update mechanisms are designed to be resilient to varying clients dataset sizes and imbalances. Instead of requiring each client to train on its entire dataset at once, *Stratify* manages the varying number of clients as a dynamic pool of available clients for each placeholder, in which the server can strategically select clients to perform local training on the masked label that a training task requires each time. In this way, *Stratify* ensures that the contribution of client with small data is meaningfully integrated by accumulating its summed gradients as part of the contribution for a batch or directly incorporating its update into the single-sample learning model then only passing the model to the next available client.

5.5. FedTAN vs. Stratify performance with BN

The performance comparison between FedTAN and *Stratify* is separately conducted using ResNet-20 on the CIFAR-10 dataset, as FedTAN's custom BN code is only available for this model architecture. Moreover, its implementation heavily relies on numerous local arrays passing and computations, making it challenging to decouple and adapt to other model architectures. In Fig. 4, we can observe that FedTAN struggles to achieve high accuracy across various non-IID settings and converges significantly slower than *Stratify*. This result indicates that merely addressing the batch statistics deviation is insufficient in a non-IID setting as it does not mitigate the underlying data distribution shift across clients. In contrast, since *Stratify* has the mechanisms that effectively prevent the adverse effects of non-IID data, the use of our custom BN layer in *Stratify* does not suffer from the same convergence issue observed in FedTAN.

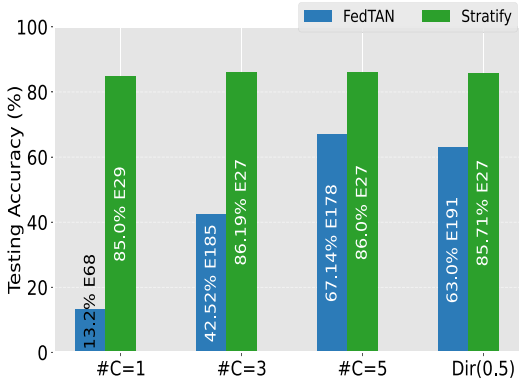


Fig. 4. FedTAN and *Stratify* performance on CIFAR-10 across various non-IID settings.

5.6. Algorithm efficiency and practical deployment

Training Overhead: As summarized in Table 3, while *Stratify* learning approach increases the total training time and communication overhead per round due to its iterative, fine-grained learning using SLS. The number of rounds required for convergence using *Stratify* is markedly lower, with competitive local training workload per client compared to conventional methods. As more clients participate, the amount of gradient computation and model update contribution required from each client per round decreases, resulting in lower overall local training time across all rounds. In the batch learning setting, the presence of more clients further accelerates training, as batch gradient computations can be performed in parallel across clients, reducing overall per-round training time. Previous research on parallel machine learning [38] highlights the role of distributed computation in improving training efficiency, reinforcing the value of distributing workload across clients to accelerate training and reduce client-side computation in federated learning.

On CIFAR-10, *Stratify* converges in just 8 rounds with an overall local training time of 546 s per client—compared to FedAvg, which takes 50 rounds, with an overall local training time of 575 s. The local training workload further amplifies with multiple local epochs in

Table 3
Total Training Time + Parameters/Gradients Serialization Deserialization and Weight Loading Time Per Round (Second) on CIFAR-10 with 10 Clients.

Algorithm	Single sample learning	Batch learning (No BN)	Batch learning (With BN)
FedAvg	53 s + ~0.006 s	11.4 s + ~0.127 s	11.5 s + ~0.154 s
FedProx	53.4 s + ~0.006 s	9.83 s + ~0.127 s	10.1 s + ~0.154 s
Scaffold	54.6 s + ~0.006 s	10.43 s + ~0.127 s	10.8 s + ~0.154 s
SFL	142 s + ~0.06 s	–	–
FedTAN	–	–	56 s + ~0.154 s
Stratify (Ours)	145 s + ~95.4 s	28.1 s + ~11.8 s	68.3 s + ~12 s

Table 4
Time (Second/Millisecond) to Construct Unique Real Labels List + Clients Label to Placeholder Mapping + Global Label Counts using Homomorphic Encryption. Each client holds #C label of 1, 5, 50 for each respective total classes.

Total class	Number of clients		
	5 clients	10 Clients	20 clients
2 Classes	20 ms + 0.8 s + 8 ms	20 ms + 0.8 s + 9 ms	9 ms + 0.9 s + 10 ms
10 Classes	0.4 s + 1.3 s + 40 ms	0.5 s + 1.7 s + 40 ms	0.6 s + 2 s + 50 ms
100 Classes	63 s + 19 s + 0.4 s	90 s + 22 s + 0.5 s	102 s + 27 s + 0.6 s

conventional methods. FedAvg's local training time increases to 1150 s with 2 local epochs and 50 training rounds. In terms of communication overhead T (see Eq. (1)), assuming a uniform update cost C_{update} of 1 per client when system heterogeneity is not considered [39], *Stratify* incurs T cost of 1000 for 8 rounds with 125 model update transfer per round, whereas FedAvg incurs T cost of 50 for 50 rounds with 1 model update transfer per round. However, although *Stratify* has a higher communication cost than FedAvg, it converges faster and achieves stronger accuracy across different non-IID data settings with minimal per-client training effort.

Secure Client Selection with Masked Label Representation: Our secure client selection protocol employs a masked label representation mechanism enabled by the CKKS homomorphic encryption scheme. Experimental evaluation (see Table 4) indicates that the overhead introduced by this privacy-preserving measure is modest and does not hinder the overall convergence of *Stratify*. The time to construct encrypted unique label list increases as the total number of classes grows, as more label vectors need to be processed by server. However, this is a one-time process. Once the unique label list reaches the required total class number, the server stops processing label vectors from any remaining clients to avoid redundant computations. The time for mapping encrypted real labels to placeholders for each client remains similar across different client numbers with concurrent processing.

Table 5
Testing Accuracy Obtained Using Uniform (U) and Weighted (W) Client Selection for 10 Clients. E# denotes the epoch *Stratify* achieves the highest accuracy, CS# denotes the placeholder chunk size used for single-sample selection.

Setting	CIFAR-10	PACS	Digits-DG
Single-Sample, U, CS5	80.4%, E28	95.2%, E15	82.5%, E13
Single-Sample, U, CS50	80.2%, E29	94.9%, E15	81.5%, E10
Single-Sample, U, CS150	80.1%, E26	94.9%, E15	80.8%, E15
Single-Sample, W, CS5	80.2%, E30	95.4%, E11	81.1%, E10
Single-Sample, W, CS50	80.7%, E26	95.5%, E15	81.3%, E14
Single-Sample, W, CS150	80.1%, E26	95.4%, E14	81.1%, E14
Batch-Data, U	90.4%, E8	**93.7%, E18**	**84.6%, E11**
Batch-Data, W	90.3%, E8	94.4%, E9	85.2%, E9

Table 6
Testing Accuracy Obtained Using ResNet-9 with Group Normalization (GN) layer for 10 Clients.
E# denotes the epoch *Stratify* achieves the highest accuracy.

Setting	CIFAR-10	CIFAR-100
Single-Sample Learning	85.1%, E8	61.9%, E30
Batch-Data Learning	83%, E8	66.1%, E38

5.7. Ablation studies and further analysis

We conducted non-IID experiments on CIFAR-10 using Dirichlet-based label distribution, and on the PACS and Digits-DG datasets with one of the domains concentrated on a single client. These setups are used to evaluate *Stratify* performance under both uniform and weighted client selection strategies, and to examine the effect of varying chunk size in the single-sample learning setting.

As shown in Table 5, in CIFAR-10, *Stratify* performs equally well with both uniform and weighted client selection, as the effects of skewed label distributions are mitigated with the stratified labels in SLS. Chunk size 5 results in around 35% reduction in model transfer frequency, compared to naive random passing for each label in SLS. Increasing the chunk size provides only an additional few percent reduction. In the non-IID setup for PACS and Digits-DG, uniform client selection leads to training being dominated in later training iterations by this single client that holds a large single domain data. Despite this, the single-sample learning setting demonstrates resilience, achieving accuracies comparable to weighted selection, especially with a smaller chunk size. The batch learning setting, on the other hand, shows slower convergence with uniform selection, but is still able to achieve accuracy close to weighted selection. Hence, our experimental results and ablation study confirm that the secure uniform client selection is effective for most non-IID scenarios, highlighting the robustness and practicality of our approach in handling a variety of data distributions, when stringent label security is required.

In addition, we further examined the single-sample learning setting using a more complex model architecture: ResNet-9 with Group Normalization (GN) [40]. This experiment aimed to confirm whether the single-sample learning, previously evaluated with simpler CNN models in the main experiments, could also deliver strong performance on a deeper architecture. As shown in Table 6, ResNet-9 with GN outperformed the simpler CNN models on CIFAR-10 and CIFAR-100 in the single-sample learning setting, highlighting the adaptability of the learning setting to various model architectures.

5.8. Discussion summary

The experimental results provide compelling evidence that *Stratify* effectively addresses the challenges posed by non-IID data in federated learning. By integrating the SLS, secure label-aware client selection, and novel single-sample and batch-data learning, *Stratify* achieves robust global updates, rapid convergence, and high accuracy under diverse conditions. Moreover, our communication efficiency analysis demonstrates that the additional privacy-preserving measures and frequent updates are balanced by a reduction in the total number of training rounds. This evidence strongly supports that *Stratify* offers a practical and effective solution to the inherent challenges of non-IID federated learning.

6. Conclusion

In this work, we demonstrate that addressing the design limitations of FedAvg through our novel guided training mechanism enables global model to effectively learn across various non-IID settings. While *Stratify* incurs high communication overhead due to frequent model updates, it is able to consistently maintain and achieve high testing accuracy across all non-IID scenarios with fewer training rounds compared to all baselines. For future work, we outline several key directions. First, enhancing the communication efficiency of Stratify through techniques such as model compression and parameter-efficient training remains an important objective to reduce overhead while preserving convergence behavior. Second, mechanisms for handling client dropout should be investigated. Third, although this work adopts uniform and weighted client selection, future extensions could explore adaptive client participation strategies that dynamically balance fairness and representation diversity. Fourth, extending *Stratify* toward personalized FL. Fifth, explore alternative global learning rebalancing methods that can effectively handle non-IID while jointly optimizing communication and computation efficiency. Lastly, as this work primarily introduces a novel training approach for robust learning in non-IID settings, we leave security challenges such as data poisoning and model inversion attacks as another critical future research direction. Specifically, the sequential nature of single-sample learning may increase susceptibility to model inversion attacks when a client contributes very few (e.g., 1 to 2) samples in a low-client environment, where a malicious client is more likely to receive the model directly after a target client updates it, thereby gaining access to consecutive model states and potentially exploiting them to reconstruct the target's private training samples. Potential mitigation to be explored include adding controlled noise to model updates or implementing sequential client selection constraints that prevent a client from observing consecutive model states. Together, these advancements could further enhance the real-world applicability of the proposed approach, and pave the way for more effective, efficient, and secure federated learning in non-IID settings.

CRediT authorship contribution statement

Hui Yeok Wong: Writing – original draft, Methodology, Conceptualization. **Chee Kau Lim:** Writing – review & editing, Supervision, Conceptualization. **Chee Seng Chan:** Writing – review & editing, Supervision, Conceptualization.

Declaration of competing interest

Chee Seng, Chan is an Associate Editor for this journal and was not involved in the editorial review or the decision to publish this article. The other authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the Fundamental Research Grant Scheme (FRGS/1/2024/ICT02/UM/01/1) from the Ministry of Higher Education, Malaysia, Malaysia. It also benefited from computational resources provided by the Data-Intensive Computing Centre, Universiti Malaya.

Data availability

The source code and data are available on GitHub at https://github.com/HuiYeok1107/Stratify.

References

[1] E. Guerra, F. Wilhelmi, M. Miozzo, P. Dini, The cost of training machine learning models over distributed data sources, IEEE Open J. Commun. Soc. 4 (2023) 1111–1126.
[2] H. Guan, P.-T. Yap, A. Bozoki, M. Liu, Federated learning for medical image analysis: A survey, Pattern Recognit. (2024) 110424.
[3] C. Briggs, Z. Fan, P. Andras, Federated learning with hierarchical clustering of local updates to improve training on non-IID data, in: 2020 International Joint Conference on Neural Networks, IJCNN, IEEE, 2020, pp. 1–9.
[4] E. Jothimurugesan, K. Hsieh, J. Wang, G. Joshi, P.B. Gibbons, Federated learning under distributed concept drift, in: International Conference on Artificial Intelligence and Statistics, PMLR, 2023, pp. 5834–5853.
[5] S. Vahidian, M. Morafah, W. Wang, V. Kungurtsev, C. Chen, M. Shah, B. Lin, Efficient distribution similarity identification in clustered federated learning via principal angles between client data subspaces, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, (8) 2023, pp. 10043–10052.
[6] Y. Xiao, J. Shu, X. Jia, H. Huang, Clustered federated multi-task learning with non-IID data, in: 2021 IEEE 27th International Conference on Parallel and Distributed Systems, ICPADS, IEEE, 2021, pp. 50–57.
[7] Y. Huang, L. Chu, Z. Zhou, L. Wang, J. Liu, J. Pei, Y. Zhang, Personalized cross-silo federated learning on non-iid data, in: Proceedings of the AAAI Conference on Artificial Intelligence, in: vol. 35, (9) 2021, pp. 7865–7873.
[8] J. Luo, S. Wu, Adapt to adaptation: Learning personalization for cross-silo federated learning, in: IJCAI: Proceedings of the Conference, vol. 2022, NIH Public Access, 2022, p. 2166.
[9] B. Ma, Y. Feng, G. Chen, C. Li, Y. Xia, Federated adaptive reweighting for medical image classification, Pattern Recognit. 144 (2023) 109880.
[10] H. Chen, A. Frikha, D. Krompass, J. Gu, V. Tresp, FRAug: Tackling federated learning with non-IID features via representation augmentation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 4849–4859.
[11] Z. Li, Y. Sun, J. Shao, Y. Mao, J.H. Wang, J. Zhang, Feature matching data synthesis for non-IID federated learning, IEEE Trans. Mob. Comput. 23 (10) (2024) 9352–9367.
[12] H. Wen, Y. Wu, J. Li, H. Duan, Communication-efficient federated data augmentation on non-iid data, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 3377–3386.
[13] T. Li, A.K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, V. Smith, Federated optimization in heterogeneous networks, Proc. Mach. Learn. Syst. 2 (2020) 429–450.
[14] A. Xu, H. Huang, Coordinating momenta for cross-silo federated learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, (8) 2022, pp. 8735–8743.
[15] S.P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, A.T. Suresh, Scaffold: Stochastic controlled averaging for federated learning, in: International Conference on Machine Learning, PMLR, 2020, pp. 5132–5143.
[16] Z. Zhao, J. Wang, W. Hong, T.Q. Quek, Z. Ding, M. Peng, Ensemble federated learning with non-IID data in wireless networks, IEEE Trans. Wirel. Commun. 23 (4) (2023) 3557–3571.
[17] N. Shi, F. Lai, R. Al Kontar, M. Chowdhury, Fed-ensemble: Ensemble models in federated learning for improved generalization and uncertainty quantification, IEEE Trans. Autom. Sci. Eng. (2023).
[18] B. McMahan, E. Moore, D. Ramage, S. Hampson, B.A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial Intelligence and Statistics, PMLR, 2017, pp. 1273–1282.
[19] S.K. Thompson, Sampling, vol. 755, John Wiley & Sons, 2012.
[20] J.D. Hamilton, Time series analysis, Princeton University Press, 2020.

[21] J.H. Cheon, A. Kim, M. Kim, Y. Song, Homomorphic encryption for arithmetic of approximate numbers, in: Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I 23, Springer, 2017, pp. 409–437.

[22] L. Deng, The MNIST database of handwritten digit images for machine learning research [best of the web], IEEE Signal Process. Mag. 29 (6) (2012) 141–142.

[23] A. Krizhevsky, G. Hinton, et al., Learning multiple layers of features from tiny images, 2009.

[24] Y. Le, X. Yang, Tiny imagenet visual recognition challenge, 2015, p. 3, CS 231N, 7, (7).

[25] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei, Imagenet: A large-scale hierarchical image database, in: 2009 IEEE Conference on Computer Vision and Pattern Recognition, Ieee, 2009, pp. 248–255.

[26] J. Blackard, Covertype, 1998, http://dx.doi.org/10.24432/C50K5N, UCI Machine Learning Repository.

[27] D. Li, Y. Yang, Y.-Z. Song, T.M. Hospedales, Deeper, broader and artier domain generalization, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 5542–5550.

[28] K. Zhou, Y. Yang, T. Hospedales, T. Xiang, Learning to generate novel domains for domain generalization, in: Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XVI 16, Springer, 2020, pp. 561–578.

[29] W. Bao, H. Wang, J. Wu, J. He, Optimizing the collaboration structure in cross-silo federated learning, in: International Conference on Machine Learning, PMLR, 2023, pp. 1718–1736.

[30] S. Yang, S. Guo, J. Zhao, F. Shen, Investigating the effectiveness of data augmentation from similarity and diversity: An empirical study, Pattern Recognit. 148 (2024) 110204.

[31] Y. Fraboni, R. Vidal, L. Kameni, M. Lorenzi, Clustered sampling: Low-variance and improved representativity for clients selection in federated learning, in: International Conference on Machine Learning, PMLR, 2021, pp. 3407–3416.

[32] D. Gao, D. Song, G. Shen, X. Cai, L. Yang, G. Liu, X. Li, Z. Wang, FedSTS: A stratified client selection framework for consistently fast federated learning, IEEE Trans. Neural Networks Learn. Syst. (2024).

[33] Y. Li, X. Lyu, Convergence analysis of sequential federated learning on heterogeneous data, Adv. Neural Inf. Process. Syst. 36 (2023) 56700–56755.

[34] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: International Conference on Machine Learning, pmlr, 2015, pp. 448–456.

[35] Y. Wang, Q. Shi, T.-H. Chang, Why batch normalization damage federated learning on non-iid data? IEEE Trans. Neural Netw. Learn. Syst. (2023).

[36] X. Qin, X. Yang, X. Tang, Practical privacy-preserving federated learning based on multiparty homomorphic encryption for large-scale models, Pattern Recognit. (2025) 112174.

[37] Q. Li, Y. Diao, Q. Chen, B. He, Federated learning on non-iid data silos: An experimental study, in: 2022 IEEE 38th International Conference on Data Engineering, ICDE, IEEE, 2022, pp. 965–978.

[38] S.A. Salman, S.A. Dheyab, Q.M. Salih, W.A. Hammood, Parallel machine learning algorithms, Mesopotamian J. Big Data 2023 (2023) 12–15.

[39] S.Q. Zhang, J. Lin, Q. Zhang, A multi-agent reinforcement learning approach for efficient client selection in federated learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, (8) 2022, pp. 9091–9099.

[40] Y. Wu, K. He, Group normalization, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 3–19.